

SR UNIVERSITY

AI ASSIST CODING

Lab-3.2

ROLL NO 2503A51L09

NAME: D.Ankitha

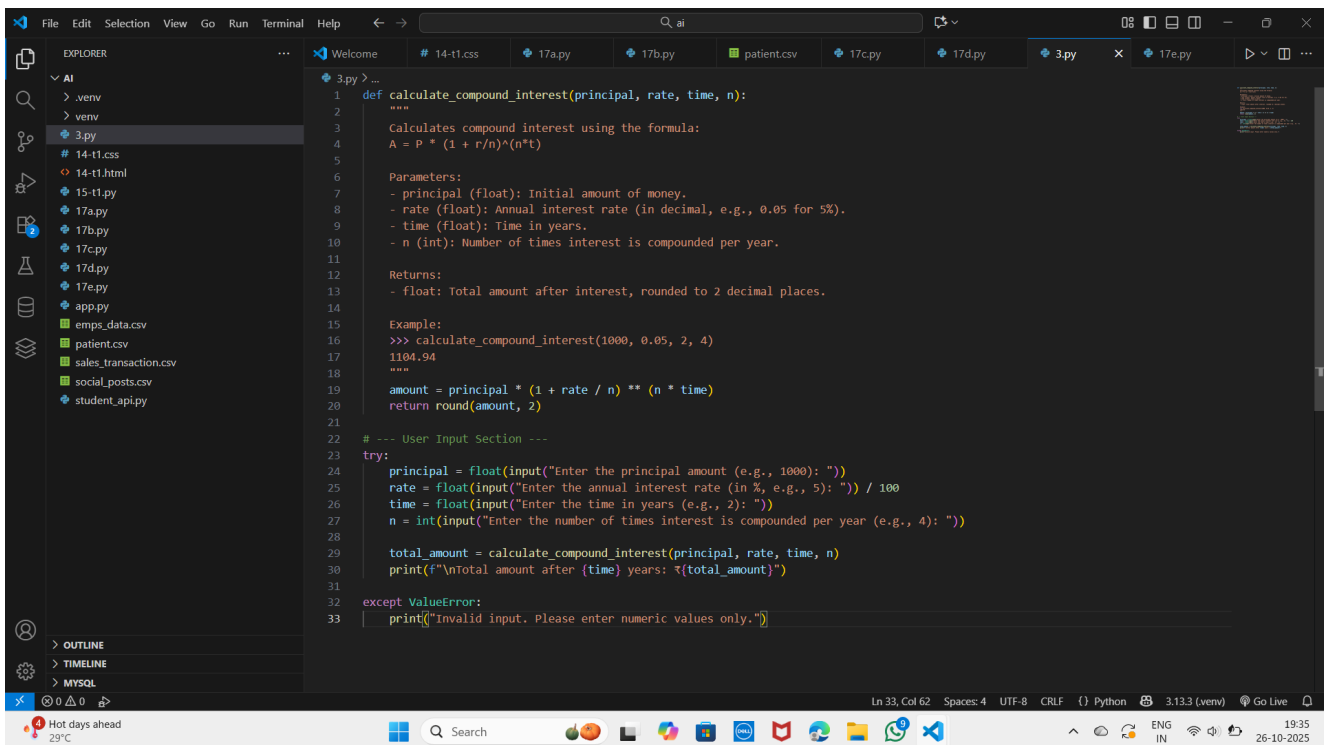
BATCH:19

TASK #1:

Prompt Used:

Ask AI to write a function to calculate compound interest, starting with only the function name. Then add a docstring, then input-output example.

Code Generated:



```
1 def calculate_compound_interest(principal, rate, time, n):
2     """
3     Calculates compound interest using the formula:
4     A = P * (1 + r/n)^(n*t)
5
6     Parameters:
7     - principal (float): Initial amount of money.
8     - rate (float): Annual interest rate (in decimal, e.g., 0.05 for 5%).
9     - time (float): Time in years.
10    - n (int): Number of times interest is compounded per year.
11
12    Returns:
13    - float: Total amount after interest, rounded to 2 decimal places.
14
15    Example:
16    >>> calculate_compound_interest(1000, 0.05, 2, 4)
17    1104.94
18    """
19    amount = principal * (1 + rate / n) ** (n * time)
20    return round(amount, 2)
21
22 # --- User Input Section ---
23 try:
24     principal = float(input("Enter the principal amount (e.g., 1000): "))
25     rate = float(input("Enter the annual interest rate (in %, e.g., 5): ") / 100)
26     time = float(input("Enter the time in years (e.g., 2): "))
27     n = int(input("Enter the number of times interest is compounded per year (e.g., 4): "))
28
29     total_amount = calculate_compound_interest(principal, rate, time, n)
30     print(f"\nTotal amount after {time} years: ₹{total_amount}")
31
32 except ValueError:
33     print("Invalid input. Please enter numeric values only.")
```

Output After executing Code:

```
PS C:\ai> & C:/ai/.venv/Scripts/Activate.ps1
(.venv) PS C:\ai> & 'c:\ai\.venv\Scripts\python.exe' 'c:\Users\ADHARSH\.vscode\extensions\ms
'52575' '--' 'c:\ai\3.py'
Enter the principal amount (e.g., 1000): 900
Enter the annual interest rate (in %, e.g., 5): 3
Enter the time in years (e.g., 2): 3
Enter the number of times interest is compounded per year (e.g., 4): 2

Total amount after 3.0 years: ₹984.1
```

Observations:

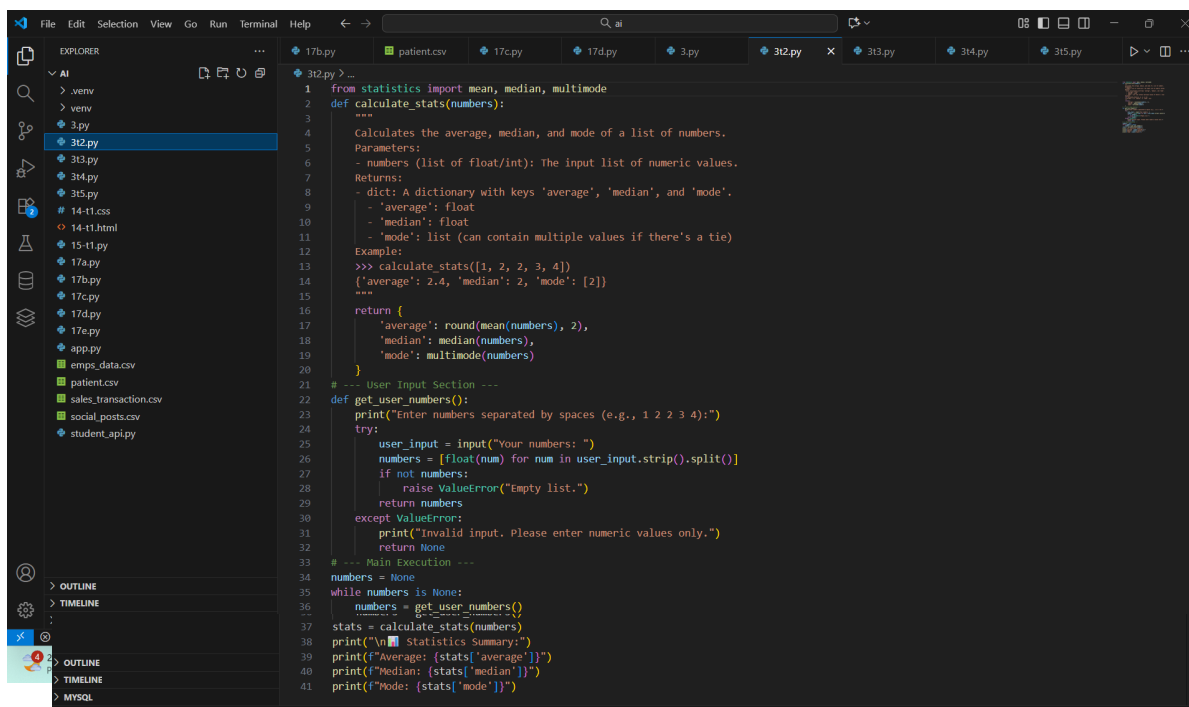
- The code correctly implements the compound interest formula using inputs (principal, rate, time, compounds per year) and returns the calculated final amount.
- The docstring explains the function with parameter details, return type, and an example, making the code clear and professional.

TASK #2:

Prompt Used:

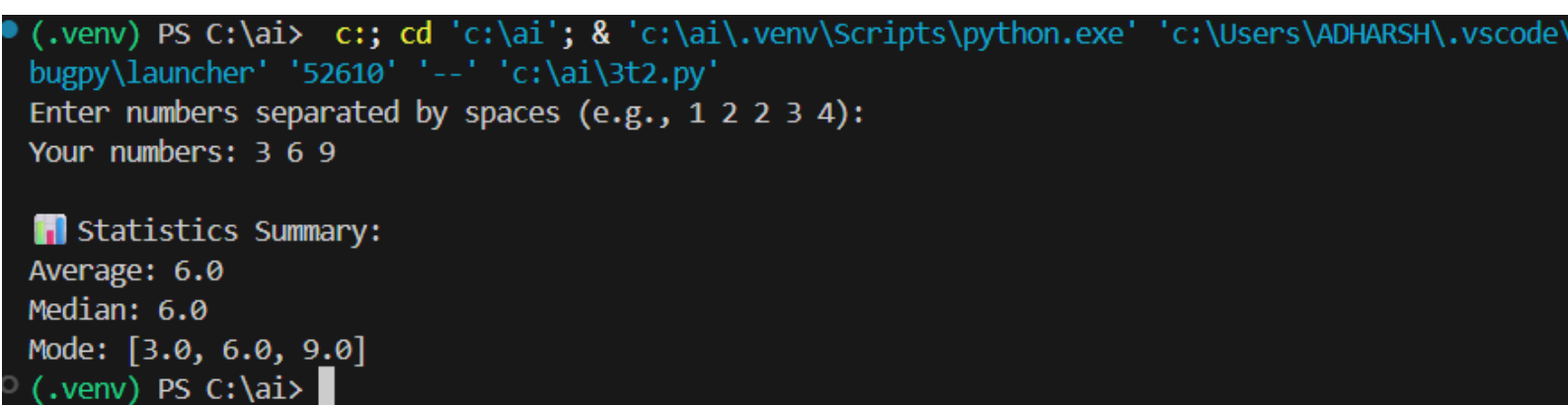
Do math stuff, then refine it to: # Write a function to calculate average, median, and mode of a list of numbers.

Code Generated:



```
1 from statistics import mean, median, multimode
2 def calculate_stats(numbers):
3     """
4     Calculates the average, median, and mode of a list of numbers.
5     Parameters:
6     - numbers (list of float/int): The input list of numeric values.
7     Returns:
8     - dict: A dictionary with keys 'average', 'median', and 'mode'.
9       - 'average': float
10      - 'median': float
11      - 'mode': list (can contain multiple values if there's a tie)
12     Example:
13     >>> calculate_stats([1, 2, 2, 3, 4])
14     {'average': 2.4, 'median': 2, 'mode': [2]}
15     """
16     return {
17         'average': round(mean(numbers), 2),
18         'median': median(numbers),
19         'mode': multimode(numbers)
20     }
21
22 # --- User Input Section ---
23 def get_user_numbers():
24     print("Enter numbers separated by spaces (e.g., 1 2 2 3 4):")
25     try:
26         user_input = input("Your numbers: ")
27         numbers = [float(num) for num in user_input.strip().split()]
28         if not numbers:
29             raise ValueError("Empty list.")
30         return numbers
31     except ValueError:
32         print("Invalid input. Please enter numeric values only.")
33         return None
34
35 # --- Main Execution ---
36 numbers = None
37 while numbers is None:
38     numbers = get_user_numbers()
39     stats = calculate_stats(numbers)
40     print("\n📊 Statistics Summary:")
41     print(f"Average: {stats['average']}")
42     print(f"Median: {stats['median']}")
43     print(f"Mode: {stats['mode']}")
```

Output After executing Code:



```
(.venv) PS C:\ai> c:: cd 'c:\ai'; & 'c:\ai\.venv\Scripts\python.exe' 'c:\Users\ADHARSH\.vscode\buggy\launcher' '52610' '--' 'c:\ai\3t2.py'
Enter numbers separated by spaces (e.g., 1 2 2 3 4):
Your numbers: 3 6 9

📊 Statistics Summary:
Average: 6.0
Median: 6.0
Mode: [3.0, 6.0, 9.0]
(.venv) PS C:\ai>
```

Observations:

- The program defines a function to calculate the average, median, and mode of a list of numbers using Python's built-in statistics functions: mean(), median(), and mode().
- A try-except block handles cases where there is no unique mode, preventing errors.

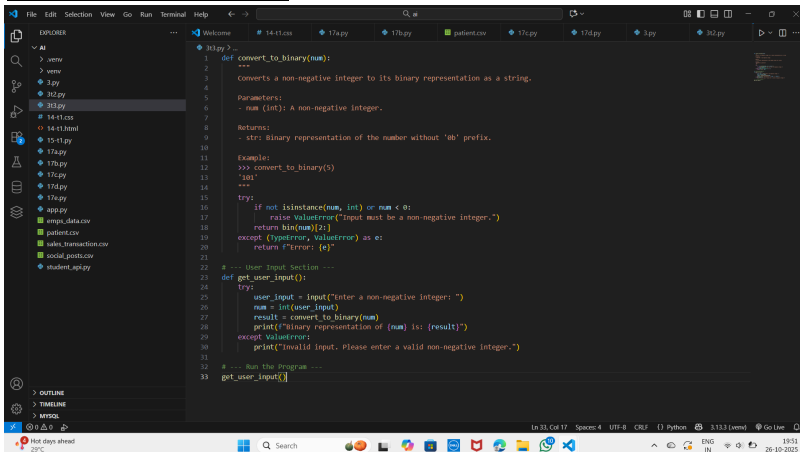
- User input is taken as a space-separated string, converted into floats, and results are displayed in a structured dictionary format.

TASK #3:

Prompt Used:

Provide multiple examples of input-output to the AI for `convert_to_binary(num)` function. Observe how AI uses few-shot prompting to generalize.

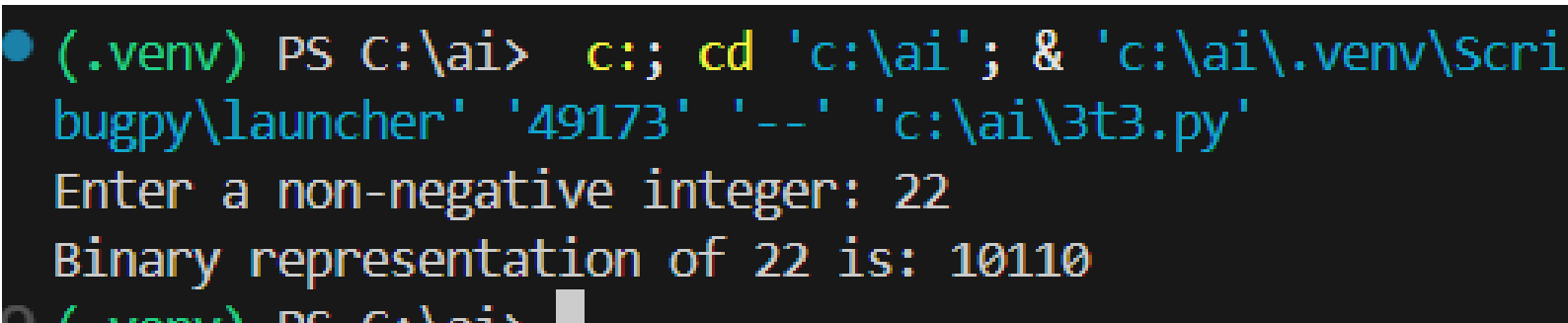
Code Generated:



```

1 def convert_to_binary(num):
2     """
3     Converts a non-negative integer to its binary representation as a string.
4
5     Parameters:
6     - num (int): A non-negative integer.
7
8     Returns:
9     - str: Binary representation of the number without '0b' prefix.
10
11     Example:
12     >>> convert_to_binary(5)
13     101
14     """
15     try:
16         if not isinstance(num, int) or num < 0:
17             raise ValueError("Input must be a non-negative integer.")
18         return bin(num)[2:]
19     except (ValueError, TypeError) as e:
20         return f"Error: {e}"
21
22 # --- User Input Section ---
23 def get_user_input():
24     try:
25         user_input = input("Enter a non-negative integer: ")
26         num = int(user_input)
27         result = convert_to_binary(num)
28         print(f"Binary representation of {num} is: {result}")
29     except ValueError:
30         print("Invalid input. Please enter a valid non-negative integer.")
31
32 # --- Run the Program ---
33 get_user_input()
  
```

Output After executing Code:



```

(.venv) PS C:\ai> c::; cd 'c:\ai'; & 'c:\ai\.venv\Scripts\bugpy\launcher' '49173' '--' 'c:\ai\3t3.py'
Enter a non-negative integer: 22
Binary representation of 22 is: 10110
  
```

Observations:

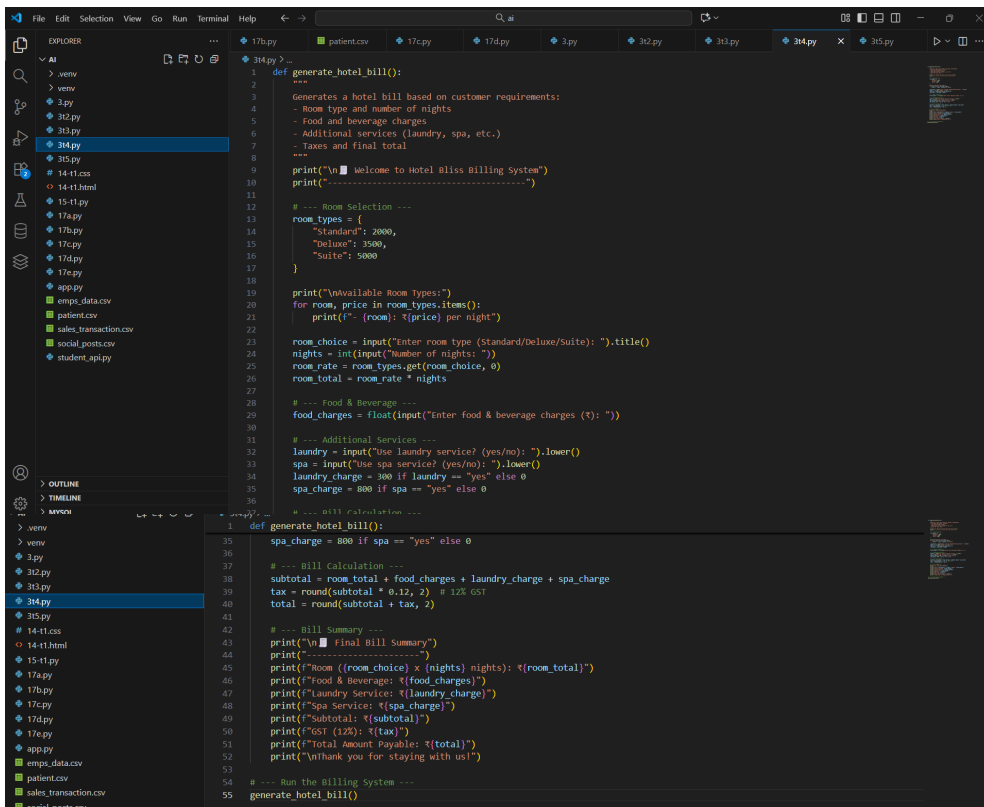
- `bin(num)` converts a decimal number to a binary string prefixed with '0b'.
- Using `[2:]` slices off the '0b' to return only the binary digits.
- The function uses a try-except block to check whether the user's input is a valid number and to handle errors if it is not.

TASK #4:

Prompt Used:

Create an user interface for an hotel to generate bill based on customer requirements

Code Generated:



```
1 def generate_hotel_bill():
2     """
3     Generates a hotel bill based on customer requirements:
4     - Room type and number of nights
5     - Food and beverage charges
6     - Additional services (laundry, spa, etc.)
7     - Taxes and final total
8     """
9     print("\n ■ Welcome to Hotel Bliss Billing System")
10    print("-----")
11
12    # --- Room Selection ---
13    room_types = {
14        "Standard": 2000,
15        "Deluxe": 3500,
16        "Suite": 5000
17    }
18
19    print("\nAvailable Room Types:")
20    for room, price in room_types.items():
21        print(f"- {room}: ₹{price} per night")
22
23    room_choice = input("Enter room type (Standard/Deluxe/Suite): ").title()
24    nights = int(input("Number of nights: "))
25    room_rate = room_types.get(room_choice, 0)
26    room_total = room_rate * nights
27
28    # --- Food & Beverage ---
29    food_charges = float(input("Enter food & beverage charges (₹): "))
30
31    # --- Additional Services ---
32    laundry = input("Use laundry service? (yes/no): ").lower()
33    spa = input("Use spa service? (yes/no): ").lower()
34    laundry_charge = 300 if laundry == "yes" else 0
35    spa_charge = 800 if spa == "yes" else 0
36
37    # --- Bill Calculation ---
38    def generate_hotel_bill():
39        spa_charge = 800 if spa == "yes" else 0
40
41        # --- Bill Calculation ---
42        subtotal = room_total + food_charges + laundry_charge + spa_charge
43        tax = round(subtotal * 0.12, 2) # 12% GST
44        total = round(subtotal + tax, 2)
45
46        # --- Bill Summary ---
47        print("\n ■ Final Bill Summary")
48        print("-----")
49        print(f"Room ({room_choice} x {nights} nights): ₹{room_total}")
50        print(f"Food & Beverage: ₹{food_charges}")
51        print(f"Laundry Service: ₹{laundry_charge}")
52        print(f"Spa Service: ₹{spa_charge}")
53        print(f"Subtotal: ₹{subtotal}")
54        print(f"GST (12%): ₹{tax}")
55        print(f"Total Amount Payable: ₹{total}")
56        print("\nThank you for staying with us!")
57
58    # --- Run the Billing System ---
59    generate_hotel_bill()
```

Output After executing Code:

```
Enter food & beverage charges (₹): 8000
Use laundry service? (yes/no): yes
Use spa service? (yes/no): no
```

```
Final Bill Summary
-----
Room (Deluxe x 3 nights): ₹10500
Food & Beverage: ₹8000.0
Laundry Service: ₹300
Spa Service: ₹0
Subtotal: ₹18800.0
```

Observations:

- The menu is stored as a dictionary with items as keys (capitalized) and their prices as values.
- User input is processed with .capitalize() so it matches the menu keys, making input case-insensitive for practical purposes.
- Separate functions are defined for repeated tasks like displaying the menu and generating the bill, ensuring code reusability.

TASK #5:

Prompt Used:

Analyzing Prompt Specificity: Improving Temperature Conversion Function with Clear Instructions

Code Generated:

```
1 def convert_temperature(value, from_unit, to_unit):
2     """
3     Converts temperature between Celsius, Fahrenheit, and Kelvin.
4
5     Parameters:
6     - value (float): Temperature value to convert.
7     - from_unit (str): Original unit ('C', 'F', 'K').
8     - to_unit (str): Target unit ('C', 'F', 'K').
9
10    Returns:
11    - float: Converted temperature rounded to 2 decimal places.
12
13    Example:
14    >>> convert_temperature(100, 'C', 'F')
15    212.0
16    """
17    try:
18        from_unit = from_unit.upper()
19        to_unit = to_unit.upper()
20
21        if from_unit == to_unit:
22            return round(value, 2)
23
24        # Convert to Celsius
25        if from_unit == 'F':
26            value = (value - 32) * 5 / 9
27        elif from_unit == 'K':
28            value = value - 273.15
29        elif from_unit != 'C':
30            raise ValueError("Invalid source unit.")
31
32        # Convert from Celsius to target
33        if to_unit == 'F':
34            result = (value * 9 / 5) + 32
35        elif to_unit == 'K':
36            result = value + 273.15
37        elif to_unit != 'C':
38            raise ValueError("Invalid target unit.")
39
40        return round(result, 2)
41    except Exception as e:
42        return f"Error: {e}"
43
44 # --- User Input Interface ---
45 def get_user_temperature():
46     print("\n🌡️ Temperature Converter")
47     print("-----")
48     try:
49         value = float(input("Enter temperature value: "))
50         from_unit = input("Convert from (C/F/K): ").strip()
51         to_unit = input("Convert to (C/F/K): ").strip()
52
53         result = convert_temperature(value, from_unit, to_unit)
54         print(f"✅ Converted temperature: {result} {to_unit.upper()}")
55     except ValueError:
56         print("❌ Invalid input. Please enter a numeric temperature and valid units.")
57
58 # --- Run the Program ---
59 get_user_temperature()
```

```
1 def convert_temperature(value, from_unit, to_unit):
2     """
3     Converts temperature between Celsius, Fahrenheit, and Kelvin.
4
5     Parameters:
6     - value (float): Temperature value to convert.
7     - from_unit (str): Original unit ('C', 'F', 'K').
8     - to_unit (str): Target unit ('C', 'F', 'K').
9
10    Returns:
11    - float: Converted temperature rounded to 2 decimal places.
12
13    Example:
14    >>> convert_temperature(100, 'C', 'F')
15    212.0
16    """
17    try:
18        from_unit = from_unit.upper()
19        to_unit = to_unit.upper()
20
21        if from_unit == to_unit:
22            return round(value, 2)
23
24        # Convert to Celsius
25        if from_unit == 'F':
26            value = (value - 32) * 5 / 9
27        elif from_unit == 'K':
28            value = value - 273.15
29        elif from_unit != 'C':
30            raise ValueError("Invalid source unit.")
31
32        # Convert from Celsius to target
33        if to_unit == 'F':
34            result = (value * 9 / 5) + 32
35        elif to_unit == 'K':
36            result = value + 273.15
37        elif to_unit != 'C':
38            raise ValueError("Invalid target unit.")
39
40        return round(result, 2)
41    except Exception as e:
42        return f"Error: {e}"
43
44 # --- User Input Interface ---
45 def get_user_temperature():
46     print("\n🌡️ Temperature Converter")
47     print("-----")
48     try:
49         value = float(input("Enter temperature value: "))
50         from_unit = input("Convert from (C/F/K): ").strip()
51         to_unit = input("Convert to (C/F/K): ").strip()
52
53         result = convert_temperature(value, from_unit, to_unit)
54         print(f"✅ Converted temperature: {result} {to_unit.upper()}")
55     except ValueError:
56         print("❌ Invalid input. Please enter a numeric temperature and valid units.")
57
58 # --- Run the Program ---
59 get_user_temperature()
```

Output After executing Code:

```
(.venv) PS C:\ai> c:: cd 'c:\ai'; & 'c:\ai\.venv\Scripts\python.exe' 'c:\Users\ADHARSH\.vscode\extensions\ms-python.debugpy-2025.14.1-w
bugpy\launcher' '61521' '---' 'c:\ai\3t5.py'
🌡️ Temperature Converter
-----
Enter temperature value: 32
Convert from (C/F/K): F
Convert to (C/F/K): K

✅ Converted temperature: 273.15 K
```

Observations:

Prompt-1: Temperature Conversion: Basic Version

- Handles only Celsius → Fahrenheit conversion
- Code is simple, minimal, and easy to follow
- No error handling

Prompt-2: Temperature Conversion: Enhanced Version

- Converts both Celsius ↔ Fahrenheit based on user input unit.
- Includes docstring, comments, and validation for invalid inputs.
- Robust, readable, and user-friendly.